第29课 递归思想（一）

> 本讲义配合课堂例题讲解使用，代码可直接在 Dev-C++ 中运行验证。

五、课堂例题

例题 1：倒计时（从 N 数到 1）

题目：输入一个正整数 N（1 ≤ N ≤ 100），用递归函数输出从 N 到 1 的倒计时，每个数字占一行。

讲解思路：
- **做什么**：输出当前的数字 n。
- **递归调用**：调用自己，参数变成 n-1（问题缩小了）。
- **终止条件**：当 n == 0 时，不再输出，直接 return。

```
#include <iostream>
using namespace std;

void countdown(int n) {
 if (n == 0) { // ③ 终止条件：数到0就停
 return;
 }
 cout << n << endl; // ① 做什么：输出当前数字
 countdown(n - 1); // ② 递归调用：数字缩小1
}

int main() {
 int n;
 cin >> n;
 countdown(n);
 return 0;
}
```

输入输出示例：

```
输入：5
输出：
5
4
3
2
1
```

板书：调用栈图

```
main() 调用 countdown(5)
│
├─ countdown(5): 输出5, 调用 countdown(4)
│ ├─ countdown(4): 输出4, 调用 countdown(3)
│ │ ├─ countdown(3): 输出3, 调用 countdown(2)
│ │ │ ├─ countdown(2): 输出2, 调用 countdown(1)
│ │ │ │ ├─ countdown(1): 输出1, 调用 countdown(0)
│ │ │ │ │ └─ countdown(0): n==0, return!
│ │ │ │ └─ 返回
│ │ │ └─ 返回
│ │ └─ 返回
│ └─ 返回
└─ 返回
```

例题 2：递归求和 1+2+…+N

题目：输入正整数 N（1 ≤ N ≤ 1000），用递归函数求 1+2+3+…+N 的值并输出。

讲解思路：
- **做什么**：当前这一层负责把 n 加上去。
- **递归调用**：`sum(n-1)` 算出 1+2+…+(n-1) 的结果。
- **终止条件**：`sum(1) = 1`，只剩一个数，直接返回。
- **递归公式**：`sum(n) = n + sum(n-1)`

```
#include <iostream>
using namespace std;

int sum(int n) {
 if (n == 1) { // ③ 终止条件：只有1个数
 return 1;
 }
 return n + sum(n - 1); // ① 做什么 + ② 递归调用
}

int main() {
 int n;
 cin >> n;
 cout << sum(n) << endl;
 return 0;
}
```

输入输出示例：

```
输入：5
输出：15
```

板书：展开过程（带返回值）

```
sum(5) = 5 + sum(4)
 sum(4) = 4 + sum(3)
 sum(3) = 3 + sum(2)
 sum(2) = 2 + sum(1)
 sum(1) = 1 到底了!
 sum(2) = 2 + 1 = 3 回来了
 sum(3) = 3 + 3 = 6
 sum(4) = 4 + 6 = 10
sum(5) = 5 + 10 = 15
```

> 重点引导学生观察：递归是"先往下钻，到底之后再一层一层把答案带回来"。

例题 3：正序输出每一位数字

题目：输入一个正整数 N（1 ≤ N ≤ 1000000000），用递归函数从高位到低位依次输出它的每一位数字，每位数字之间用空格隔开。

讲解思路：
- 如果用循环，从低位取容易（% 10），但从高位开始输出就麻烦了。
- 递归的妙处：**先递归（处理前面的高位），回来之后再输出当前最低位**。
- 但这题我们换个思路：**先递归处理 n/10（去掉最后一位），再输出 n%10（最后一位）**。
- **终止条件**：n 是个位数（n < 10）时，直接输出。

```
#include <iostream>
using namespace std;

void printDigits(int n) {
 if (n < 10) { // ③ 终止条件：只剩一位数
 cout << n;
 return;
 }
 printDigits(n / 10); // ② 递归调用：先输出前面的高位
 cout << " " << n % 10; // ① 做什么：输出当前最低位
}

int main() {
 int n;
 cin >> n;
 printDigits(n);
 cout << endl;
 return 0;
}
```

输入输出示例：

```
输入：2048
输出：2 0 4 8
```

板书：调用栈图

```
printDigits(2048)
 先调用 printDigits(204)
 先调用 printDigits(20)
 先调用 printDigits(2)
 2 < 10, 输出 "2", return
 输出 " 0" (20 % 10 = 0)
 输出 " 4" (204 % 10 = 4)
 输出 " 8" (2048 % 10 = 8)

最终输出：2 0 4 8
```

> 引导学生体会：**递归调用放在输出前面**，就能实现"先处理高位，后处理低位"——这是递归的一个巧妙用法！

例题 4（拓展）：递归判断回文串

题目：输入一个只含小写字母的字符串 s（长度 ≤ 100），用递归判断它是否是回文串，输出 `Yes` 或 `No`。

讲解思路：
- 回文串：正读反读都一样，如 "abcba"。
- 递归思路：比较**第一个字符**和**最后一个字符**——
- 如果不一样 不是回文，直接返回 false。
- 如果一样 把"头尾去掉"，剩下的部分交给递归去判断。
- **终止条件**：字符串长度 ≤ 1 时，一定是回文（空串或单个字符）。

```
#include <iostream>
#include <string>
using namespace std;

bool isPalindrome(string s, int left, int right) {
 if (left >= right) { // ③ 终止条件：检查完了
 return true;
 }
 if (s[left] != s[right]) { // 头尾不一样，不是回文
 return false;
 }
 // ② 递归调用：去掉头尾，检查中间部分
 return isPalindrome(s, left + 1, right - 1);
}

int main() {
 string s;
 cin >> s;
 if (isPalindrome(s, 0, s.size() - 1)) {
 cout << "Yes" << endl;
 } else {
 cout << "No" << endl;
 }
 return 0;
}
```

输入输出示例：

```
输入：abcba
输出：Yes

输入：hello
输出：No
```

板书：调用栈图（以 "abcba" 为例）

```
isPalindrome("abcba", 0, 4) a==a
 isPalindrome("abcba", 1, 3) b==b
 isPalindrome("abcba", 2, 2) left>=right return true!
 return true
 return true
```

> 这道题展示了递归在字符串问题上的妙用。和之前学的字符串知识结合起来了！

附：例题源代码与运行结果

例题 1：Countdown (N to 1)

```
// Input a positive integer N, use recursion to print
// numbers from N down to 1, one per line.
#include <iostream>
using namespace std;

void countdown(int n) {
 if (n == 0) { // base case: stop at 0
 return;
 }
 cout << n << endl; // do: print current number
 countdown(n - 1); // recurse: shrink by 1
}

int main() {
 int n;
 cin >> n;
 countdown(n);
 return 0;
}
```

Sample Input

```
5
```

Expected Output

```
5
4
3
2
1
```

例题 2：Recursive Sum 1+2+...+N

```
// Input a positive integer N, use recursion to compute
// and output 1+2+3+...+N.
#if 0
#include <iostream>
using namespace std;

int sum(int n) {
 if (n == 1) { // base case: only one number
 return 1;
 }
 return n + sum(n - 1); // recursive formula
}

int main() {
 int n;
 cin >> n;
 cout << sum(n) << endl;
 return 0;
}
#endif
```

Sample Input

```
5
```

Expected Output

```
15
```

例题 3：Print Digits in Order

```
// Input a positive integer N, use recursion to print
// its digits from most significant to least significant,
// separated by spaces.
#if 0
#include <iostream>
using namespace std;

void printDigits(int n) {
 if (n < 10) { // base case: single digit
 cout << n;
 return;
 }
 printDigits(n / 10); // recurse: handle higher digits first
 cout << " " << n % 10; // then print current last digit
}

int main() {
 int n;
 cin >> n;
 printDigits(n);
 cout << endl;
 return 0;
}
#endif
```

Sample Input

```
2048
```

Expected Output

```
2 0 4 8
```

例题 4：Recursive Palindrome Check

```
// Input a lowercase string, use recursion to determine
// if it is a palindrome. Output "Yes" or "No".
#if 0
#include <iostream>
#include <string>
using namespace std;

bool isPalindrome(string s, int left, int right) {
 if (left >= right) { // base case: checked all pairs
 return true;
 }
 if (s[left] != s[right]) { // mismatch found
 return false;
 }
 // recurse: strip head and tail, check inner part
 return isPalindrome(s, left + 1, right - 1);
}

int main() {
 string s;
 cin >> s;
 if (isPalindrome(s, 0, s.size() - 1)) {
 cout << "Yes" << endl;
 } else {
 cout << "No" << endl;
 }
 return 0;
}
#endif
```

Sample Input

```
abcba
```

Expected Output

```
Yes
```